Queen Mary
University of London
Science and Engineering

QMUL-BUPT Joint Programme/
QMUL-BUPT Joint Education Institute

EBU5477 Embedded Systems

Module Introduction
and Foundations of Embedded Systems

arm
University Program Education Kits

Welcome to the Module

- Welcome to Embedded Systems EBU5477
- One of the most important practical modules in your degree
- This module bridges:
  - Hardware
  - Software
  - Real-world engineering systems
- You will learn to design and build real embedded systems

Module Learning Outcomes

- Recognise
  - what embedded systems are and identify their applications in society.
- Become familiar
  - with the architecture and organisation of ARM Cortex-M microcontrollers (MCUs).
- Learn and practice
  - C programming techniques for Cortex-M MCUs.
- Investigate
  - the societal impacts of embedded systems in areas like sustainability and ethics.

Teaching Team

Lecturer	Email
Dr Mannan Muhammad, Module Organiser	mannan.muhammad@qmul.ac.uk
Dr Ethan Lau	ethan.lau@qmul.ac.uk

Communication Channels

- For **questions** & **discussions**, use **QMPlus message board**
- For personal issues, send emails with a proper subject **[EBU5477]**

What You Will Learn

(Expanded Learning Outcomes)

- By the end of this module, you will:
  - Identify embedded systems in real-world applications
  - Design embedded systems architecture
  - Program ARM Cortex-M using C
  - Interface sensors and actuators
  - Understand system-level trade-offs

Module Structure

- The module consists of:
  - Lectures
  - Lab Practice
  - Mini Project
  - Quizzes
  - Final Exam
- Hands-on learning is essential.

Assessment

Summative Coursework

Items	Weights	Remarks
4-Lab Practice	10%	Practical work of ARM C programming and hardware (2.5% each)
1-Mini project	10%	Integrate the skills from your lab practise to create your own “embedded system”
Quiz 1 Quiz 2	2.5% + 2.5%	Two quizzes will be held in-class at the end of Block 2 and Block 4
Exam	75%	Covers everything - lectures, labs and other coursework

All other exercises/assignments are formative and they are useful learning resources even though no marks are given.

About the Quizzes

- Two quizzes:
  - 2.5%+2.5% of total grade
  - End of Block 2
  - End of Block 4
  - Closed book
  - In-class
  - Test conceptual understanding

Lab Component

- Labs will cover:
  - 2.5%+2.5% +2.5%+2.5% of total grade
  - GPIO programming
  - ADC
  - Timers
  - Interrupts
  - Communication protocols (UART, SPI, I2C)
  - Real embedded debugging
- You are expected to attend and engage in the lab sessions.

Mini Project

You will:

- Design your own embedded system
  - 10% of total grade
- Integrate:
  - Sensors
  - Processing
  - Output/Actuation
- Demonstrate working prototype
- Focus:
  - Creativity & Reliability
  - Proper coding structure
  - Proper documentation structure

Final Exam

- 75% of total grade
- Covers:
  - Lectures
  - Labs
  - Concepts
  - Problem-solving
- Requires:
  - Understanding
  - Not memorisation

QMPlus

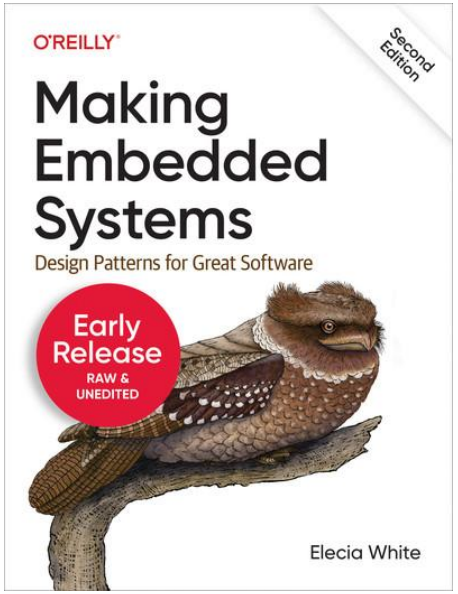
- Administration: announcements, updates, grades, etc.
- Course materials: lecture notes, quizzes, exercises
- Lab materials: software/tools, lab manuals, submissions, feedback
- Lab and Assignments: always submit online.

Please make sure that you can access the course area.
Remember to check that regularly for news/updates.

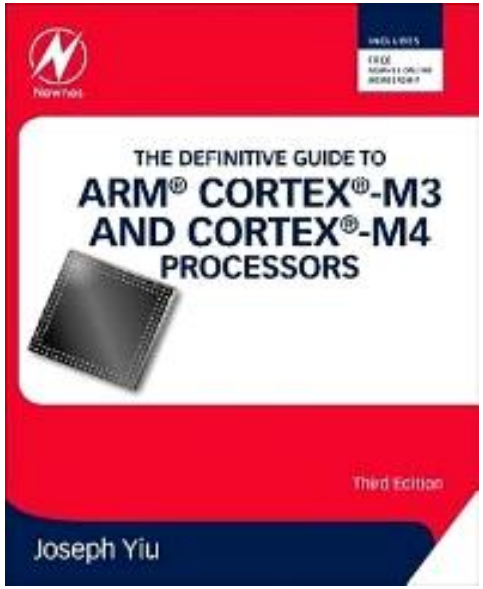
Suggested Readings

ALL available online via QMUL library website!

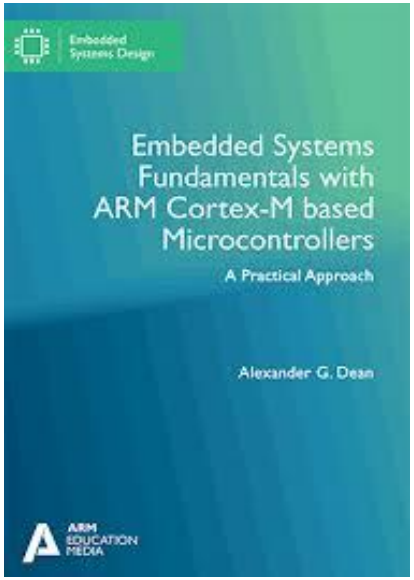
Note: There is not a single textbook that covers the various materials in this module.



Making Embedded Systems, 2nd Edition by Elecia White



The Definitive Guide to ARM® Cortex®-M3 and Cortex®-M4 Processors 3rd Edition by Joseph Yiu

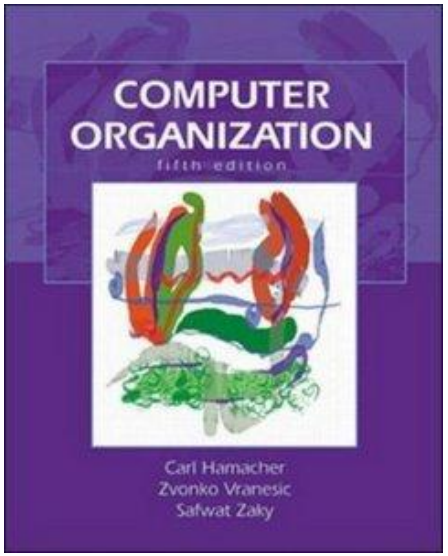


Embedded Systems Fundamentals with Arm Cortex-M Based Microcontrollers: A Practical Approach by Alexander G. Dean

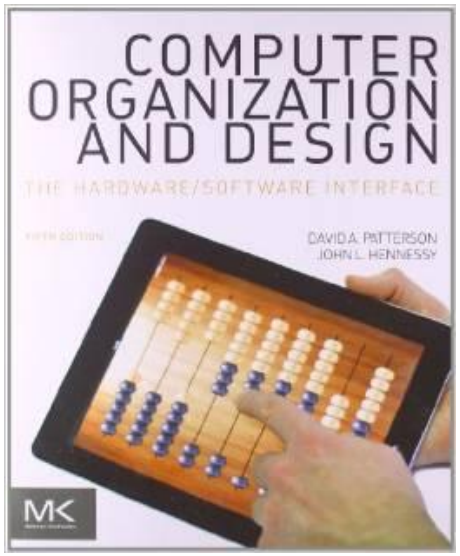
Useful References

These are also available online via QMUL library website!

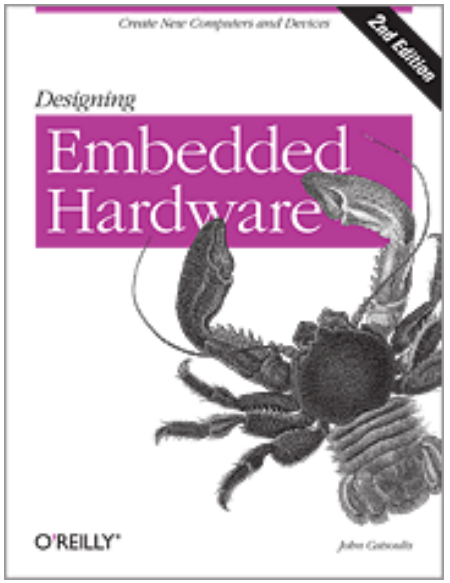
Computer Organization (5th edition)
By Carl Hamacher et. al.



Computer Organization and Design: The Hardware/Software Interface (5th edition)
By David A. Patterson, John L. Hennessy



Designing Embedded Hardware, 2nd Ed
By John Catsoulis



Expectations from You

- Attend lectures
- Complete labs
- Ask questions
- Work consistently
- Start early on mini project

Embedded systems cannot be learned last minute.

Do you have any questions?

Please make use of the QMPlus student forum / message board for questions and discussions outside face-to-face sessions.

Foundations of Embedded Systems

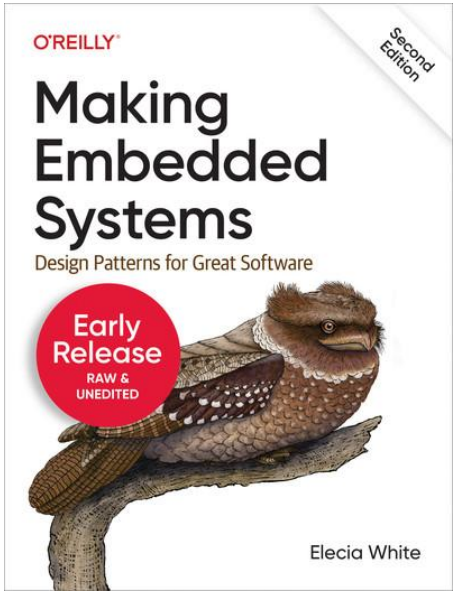
Learning Outcomes

- To define
  - embedded systems and identify design constraints associated with embedded systems
- To identify
  - core components in an embedded system and outline their roles
- To become familiar
  - with examples of embedded systems and be able to identify their key features.
- To understand
  - the concept of concurrency between hardware and software

Suggested Reading

- Making Embedded Systems – Chapter 1 (Elecia White)

- Why this reading?
  - Real-world perspective
  - Practical engineering focus
  - Industry mindset



Making Embedded Systems, 2nd Edition by Elecia White

What is an Embedded System?

- An embedded system is:
  - A computer system designed for a specific function within a larger system.

- Examples:
  - Washing machines
  - Cars
  - Smart thermostats
  - Medical devices
  - Drones

Inside a washing machine, there is a microcontroller that:

- Reads water level sensors
- Monitors temperature
- Controls motor speed
- Decides washing cycles
- Ensures safety (door lock control)

It is not a general computer — it only controls washing operations.

Modern cars contain 50–100+ embedded systems.
Examples:

- Engine Control Unit (ECU)
- ABS braking system
- Airbag controller
- Infotainment system

The Engine Control Unit:

- Monitors oxygen sensors
- Adjusts fuel injection
- Controls ignition timing

All decisions must happen in real-time for safety.

What is an Embedded System?

- An embedded system is:
  - A computer system designed for a specific function within a larger system.

- Examples:
  - Washing machines
  - Cars
  - Smart thermostats
  - Medical devices
  - Drones

A smart thermostat:
- Measures room temperature
- Compares it with desired temperature
- Controls heating system
- Connects to Wi-Fi
- Learns usage patterns

It combines sensing + computation + networking + actuation.

Medical Devices have embedded systems:
Examples:
- Insulin pumps
- Heart rate monitors
- Pacemakers

A pacemaker:
- Monitors heart rhythm
- Detects irregularities
- Sends electrical pulses
- Must operate 24/7 reliably

Safety-critical embedded system.

What is an Embedded System?

- An embedded system is:
  - A computer system designed for a specific function within a larger system.

- Examples:
  - Washing machines
  - Cars
  - Smart thermostats
  - Medical devices
  - Drones

Drones use embedded systems to:
- Stabilise flight (using IMU sensors)
- Control motor speeds
- Process GPS signals
- Avoid obstacles
- Communicate with remote controller

Real-time control with tight timing constraints.

What is an Embedded System?

An embedded system is a computerised system that is purpose-built for its application.

- An embedded system has less support for things that are unrelated to accomplishing the job at hand. The hardware often has constraints.
  - e.g. a CPU that runs more slowly to save battery power
- The embedded software must act exactly the same each time (deterministically) or in real time (always reacting to an event fast enough).
- Some systems require that the software be fault-tolerant, with graceful degradation in the face of errors. (e.g. a tracking tag on a whale)

What is an Embedded System?

An embedded system is a computerised system that is purpose-built for its application.

- Key characteristics:
  - **Dedicated function**
    - An embedded system performs a specific task rather than running many unrelated applications.
      It is optimised for one job only.
  - **Deterministic behaviour**
    - The system must behave predictably. Given the same input, it should produce the same output every time.
      Many embedded systems must respond within strict timing constraints (real-time operation).
  - **Resource constrained**
    - Embedded systems typically have limited memory, processing power, storage,
    - energy (often battery-powered).
    - The hardware is chosen carefully to reduce cost and power consumption.
  - **Often invisible to user**
    - Embedded systems usually operate in the background.
      Users interact with the device, not directly with the embedded computer inside it.

What is an Embedded System?

Design Constraints in Embedded Systems

- Embedded systems have:
  - Limited CPU speed
  - Limited memory
  - Limited power
  - Strict cost limits
  - Environmental constraints
- Unlike PCs, they are optimised for a specific job.

Determinism & Real-Time

- Embedded software must:
  - Act exactly the same way each time (deterministic)
  - Respond within deadline (real-time)
- Examples:
  - Airbag deployment
  - Motor braking
  - Medical infusion pump

What is an Embedded System?

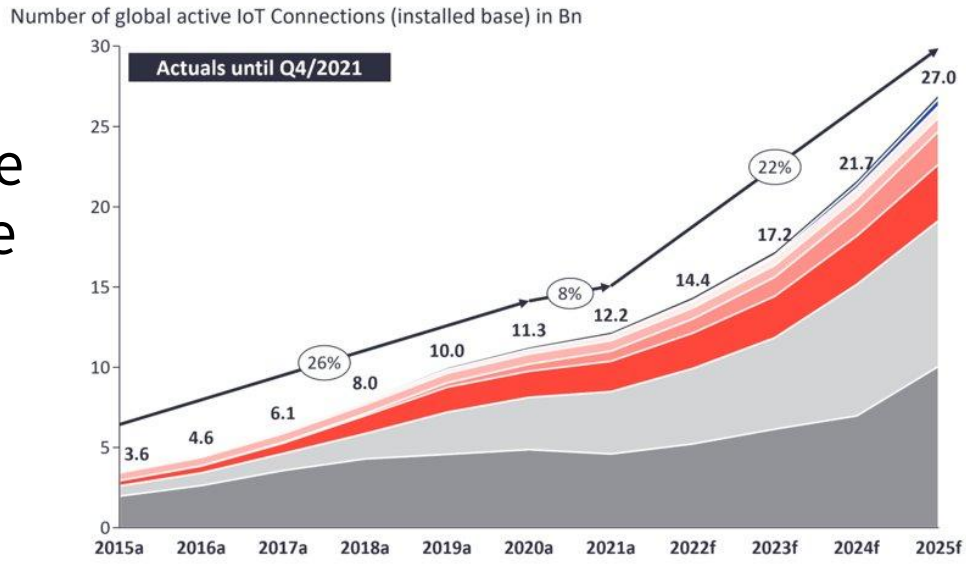
Fault Tolerance

- Some systems must:
  - Continue working despite errors
  - Fail gracefully
  - Log faults for diagnosis
- Example:
  Tracking tag on whale—Must survive harsh conditions

Embedded Systems: What do they do?

How are embedded systems used in our modern world?

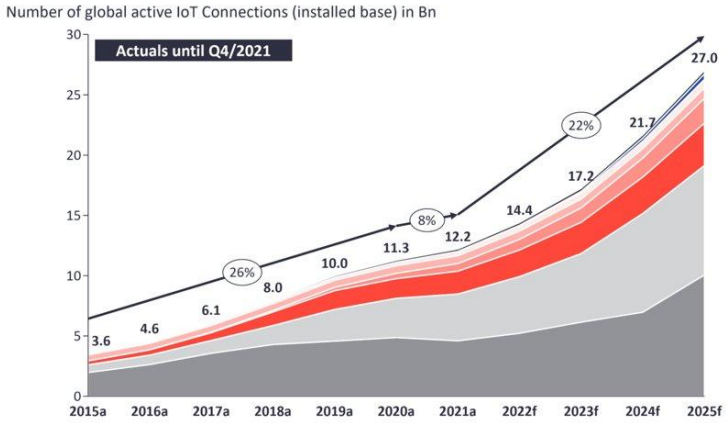
- **Data acquisition:** sensors from different environments generate a huge amount of data
- **Local computation:** we need proper filtering and some processing of data
- **Network connectivity:** the systems communicate with each other and share information.
- **Local actuation:** affect and change the physical world with messages.



source: https://www.iotforall.com/state-of-iot-2022

Growth to IoT systems

- Over the last decade, the number of connected devices worldwide has grown exponentially.
- The graph illustrates:
  - Rapid increase in global IoT connections
  - From a few billion devices in the mid-2010s
  - To tens of billions today
  - With projections continuing upward
- This growth shows one important fact:
  - Embedded systems are everywhere.
- Every connected device contains at least one embedded processor.



source: https://www.iotforall.com/state-of-iot-2022

Where Are These Devices Used? Examples

1. Smart Homes

source: https://microcontrollerslab.com/home-automation-projects-ideas/

- Examples:
  - Smart lights
  - Smart door locks
  - Smart speakers
  - Smart thermostats
- Each device:
  - Collects data (temperature, motion, sound)
  - Processes locally
  - Connects via Wi-Fi or Bluetooth
  - Controls something physical
- Multiple embedded systems communicating together.

Where Are These Devices Used? Examples

2. Industry 4.0

- Modern factories use:
  - Sensor networks
  - Robotic arms
  - Automated inspection systems
  - Predictive maintenance systems
- Embedded systems:
  - Monitor machinery
  - Detect faults
  - Optimise production
  - Reduce downtime
- Real-time control + high reliability.

source: https://www.trentonsystems.com/en-us/resource-hub/blog/edge-computing-fourth-industrial-revolution

Where Are These Devices Used? Examples

3. Healthcare Devices

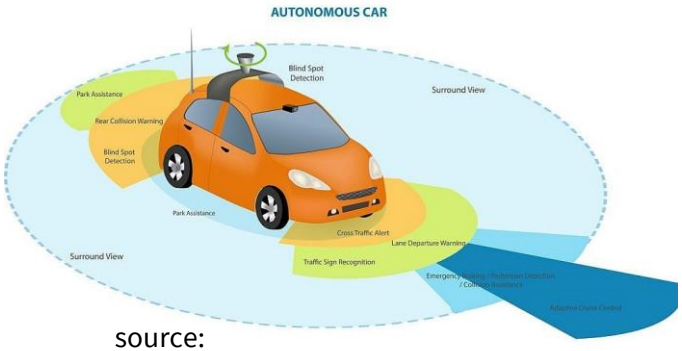
- Examples:
  - Wearable heart monitors
  - Blood oxygen sensors
  - Smart insulin pumps
  - Remote patient monitoring
- Embedded systems:
  - Collect physiological data
  - Analyse signals
  - Send alerts
  - Communicate with hospitals
- Safety-critical and power-efficient systems.

source: https://promwad.com/news/how-embedded-systems-enable-smart-medical-wearables

Where Are These Devices Used? Examples

4. Autonomous Vehicles

- Autonomous systems rely heavily on embedded processors.
- They:
  - Process camera and LiDAR data
  - Run control algorithms
  - Make split-second decisions
  - Control steering and braking
- These are highly complex, real-time embedded systems.



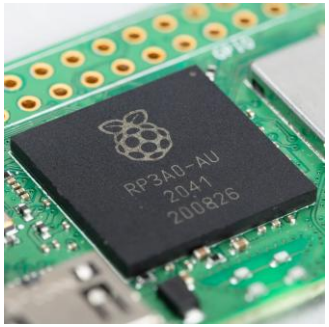
source: https://www.linkedin.com/pulse/learn-self-driving-cars-linux-embedded-iot-protocols-jci9c/

General Purpose vs Embedded Systems

General Computer	Embedded System
Multiple applications	Specific task
High performance	Low power
Large OS	Bare-metal / RTOS
User interface	Often hidden

Embedded (Micro)processors - Examples

- We use general purpose computers like desktop and laptop PCs (personal computer) a lot every day.
  - Inside, there is a microprocessor that handles the majority of the computation.
- Actually, there are many more microprocessors hidden in printers, thermostats, calculators, cars, etc. to provide intelligent control of the systems. These are called embedded microprocessors.



Raspberry Pi Zero 2 W – Pi Zero 2 W
https://shop.pimoroni.com/products/raspberry-pi-zero-2-w?variant=39493046075475

https://www.st.com/en/microcontrollers-microprocessors/stm32f401re.html

Why ARM Cortex-M?

- ARM Cortex-M microcontrollers are one of the most widely used processor families in modern embedded systems.
- **Industry Standard Architecture**
  - Adopted by major semiconductor companies (STMicroelectronics, NXP, TI, Nordic, etc.)
  - Huge ecosystem: tools, documentation, libraries, middleware
  - Strong industry demand → valuable career skill

Why ARM Cortex-M?

Widely Used Across Industries

- **Automotive**
  - Engine control units (ECU)
  - Airbag systems
  - Battery management systems
  - Dashboard instrumentation
- **Medical**
  - Portable monitoring devices
  - Infusion pumps
  - Diagnostic instruments
  - Wearable health devices
- **Robotics**
  - Motor control
  - Sensor fusion
  - Real-time control loops
  - Embedded AI at the edge
- **Consumer Electronics**
  - Smart watches
  - Remote controls
  - Smart appliances
  - IoT home devices

Why ARM Cortex-M?

Widely Used Across Industries

- **Low Power Design**
  - Designed for energy-efficient operation
  - Ideal for battery-powered systems
  - Sleep modes and power-saving features
  - Enables long device lifetime
- **High Efficiency**
  - Optimised instruction set
  - Good performance per watt
  - Hardware support for interrupts and peripherals
  - Efficient memory usage
- **Real-Time Capable**
  - Deterministic interrupt handling
  - Fast context switching
  - Suitable for time-critical applications
  - Can run bare-metal or RTOS (Real-Time Operating System)

Societal & Ethical Impact

- Embedded systems are not just technical devices
  - they directly impact society, safety, and the environment.
- As engineers, your design decisions have real-world consequences.

Societal & Ethical Impact

- Embedded systems are not just technical devices — they directly impact society, safety, and the environment.
- As engineers, your design decisions have real-world consequences.

Energy Consumption

- Billions of IoT devices consume power daily
- Poorly designed systems increase global energy demand
- Efficient firmware and low-power hardware design reduce carbon footprint
- Battery-powered systems require careful optimisation

- Small design decisions scale globally.

Societal & Ethical Impact

- Embedded systems are not just technical devices — they directly impact society, safety, and the environment.
- As engineers, your design decisions have real-world consequences.

Sustainability

- Electronic waste (e-waste) is growing rapidly
- Short product lifecycles increase environmental impact
- Engineers must consider:
  - Energy-efficient operation
  - Long-term maintainability
  - Recyclable materials
  - Firmware upgradability
- Sustainable engineering is part of modern responsibility.

Societal & Ethical Impact

- Embedded systems are not just technical devices — they directly impact society, safety, and the environment.
- As engineers, your design decisions have real-world consequences.

Data Privacy

- Embedded systems collect sensitive data:
  - Health data
  - Location data
  - Usage behaviour
- Poor design can expose personal information
- Secure storage and encrypted communication are essential
- Privacy must be designed, not added later.

Societal & Ethical Impact

- Embedded systems are not just technical devices — they directly impact society, safety, and the environment.
- As engineers, your design decisions have real-world consequences.

Security

- IoT devices are common attack targets
- Weak authentication leads to:
  - Data breaches
  - Botnets
  - Infrastructure attacks
- Secure boot, firmware validation, and encryption are critical
- Security failures can affect millions of users.

Societal & Ethical Impact

- Embedded systems are not just technical devices — they directly impact society, safety, and the environment.
- As engineers, your design decisions have real-world consequences.

AI & Autonomy

- Embedded systems increasingly make autonomous decisions
- Used in:
  - Self-driving vehicles
  - Medical devices
  - Industrial robots
- Errors can lead to physical harm
- Ethical responsibility increases with autonomy.

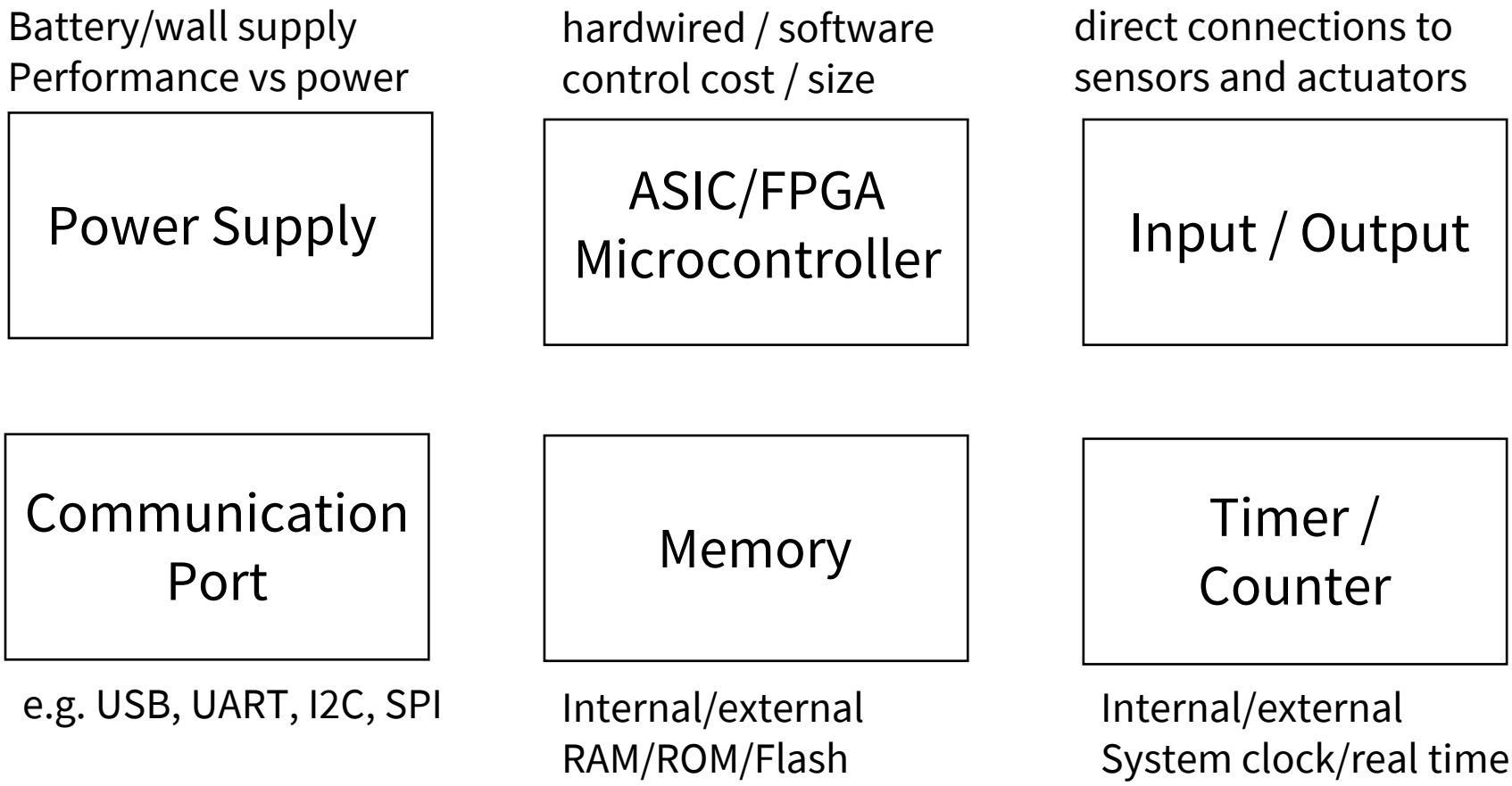
Example 1: Bike Computer

- Functions
  - Speed and distance measurement
- Constraints
  - Size
  - Cost
  - Power and Energy
  - Weight
- Inputs
  - Wheel rotation indicator
  - Mode key
- Output
  - Liquid Crystal Display
- Low performance MCU
  - 8-bit, 10 MIPS (Million Instructions Per Second)

Example 2: Motor Control Unit in Electric Vehicles

- Functions
  - Motor control
  - System communications
  - Current monitoring
  - Rotation speed detection
- Constraints
  - Reliability in harsh environment
  - Cost
  - Weight
- Many Inputs and Outputs
  - Discrete sensors & actuators
  - Network interface to rest of car
- High Performance MCU
  - 32-bit, 256 KB flash memory, 80 MHz

The Core Components in Embedded Systems

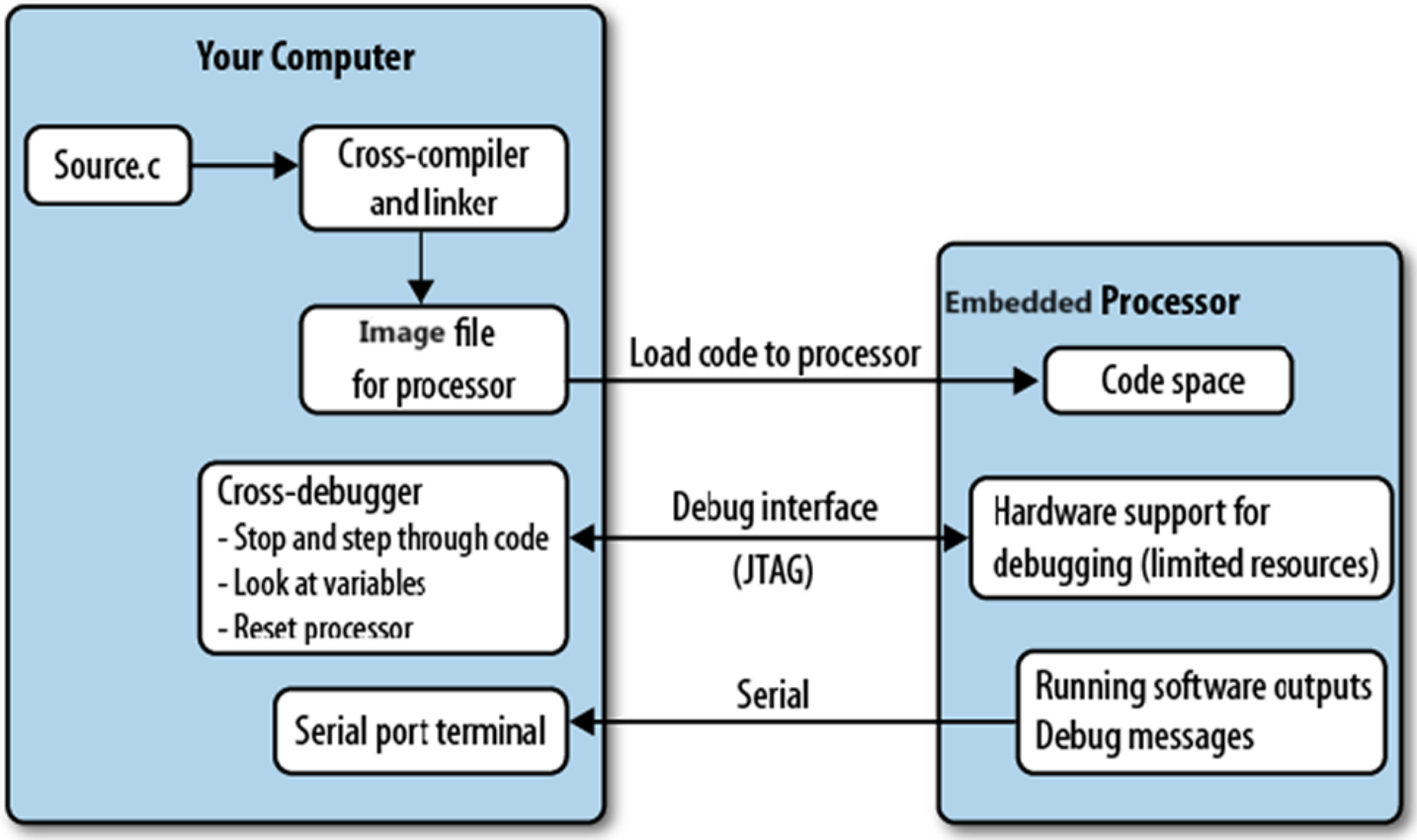


Reference: https://www.theengineeringprojects.com/2021/03/components-of-embedded-systems.html

Embedded Systems Development

- Most embedded software engineers develop a toolkit for **dealing with the constraints.**
- **Compilers and Languages**
  - Embedded systems use **cross-compilers** to generate code image to run on your target embedded system
  - Embedded software compilers often support only C, or C and C++.
- **Debugging**
  - Embedded systems use **cross-debuggers**: the debugger sits on your computer and communicates with the target processor through a special processor interface (JTAG).

Embedded Systems: Development

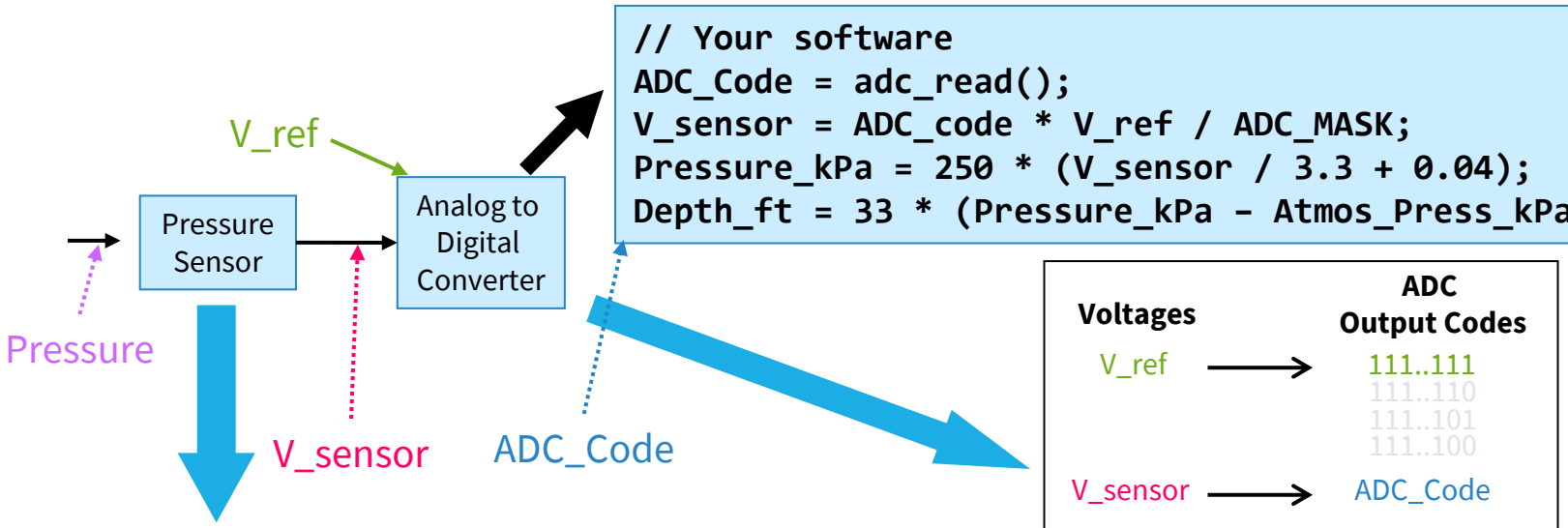


Embedded Systems: Attributes (1)

Interfacing with larger system and environment

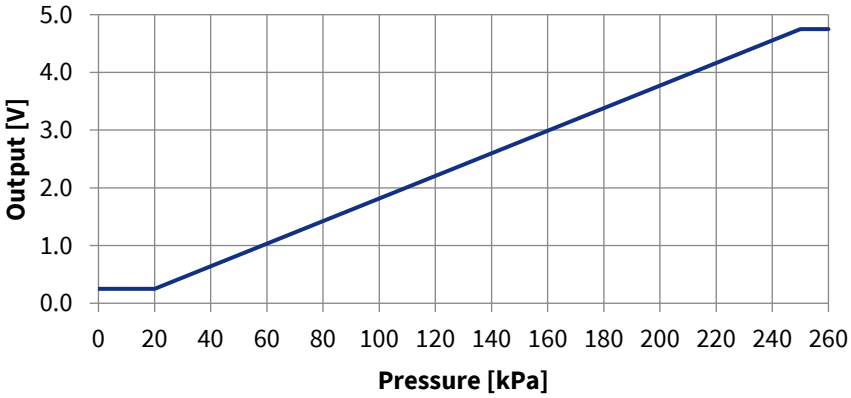
- Analog signals
  - for reading sensors
    Typically use a voltage to represent a physical value
- Power electronics
  - for driving motors
- Digital interfaces
  - for communicating with other digital devices
- Outputs
  - Simple - switches
  - Complex - displays

Example: Analog Sensor - Depth Gauge




```
// Your software
ADC_Code = adc_read();
V_sensor = ADC_code * V_ref / ADC_MASK;
Pressure_kPa = 250 * (V_sensor / 3.3 + 0.04);
Depth_ft = 33 * (Pressure_kPa – Atmos_Press_kPa) / 101.3;
```


Typical Absolute Pressure vs. Output



- Sensor detects pressure and generates a proportional output voltage V_sensor
- ADC generates a proportional digital integer (code) based on V_sensor and V_ref
- Code can convert that integer to a something more useful
  - first a float representing the voltage,
  - then another float representing pressure,
  - finally another float representing depth

Embedded Systems: Attributes (2)

Concurrent, reactive behaviours

- Continuously react to inputs from the environment by generating corresponding outputs
- Required to respond to sequences and combinations of events
- Real-time systems have deadlines on responses
  - Typically perform multiple separate activities concurrently
- Typical applications include vehicle control, consumer electronics, remote sensing, smart household appliances etc.

Embedded Systems: Reactive Behaviour

- Embedded systems are **reactive systems**.
- They do not simply execute a program and stop — they continuously interact with their environment.

- **Continuously Monitor Inputs**
  - Read sensors repeatedly
  - Detect changes in environment
  - Handle interrupts and events
  - Operate in an infinite control loop
- Example:
  - A temperature controller constantly reads the temperature sensor to decide whether heating is needed.

Embedded Systems: Reactive Behaviour

- Embedded systems are **reactive systems**.
- They do not simply execute a program and stop — they continuously interact with their environment.

- **React Immediately**
  - Respond as soon as an event occurs
  - Trigger outputs without noticeable delay
  - Often interrupt-driven
- Example:
  - When a collision sensor detects impact, the airbag system must react instantly.

Embedded Systems: Reactive Behaviour

- Embedded systems are **reactive systems**.
- They do not simply execute a program and stop —
  they continuously interact with their environment.

- **Meet Deadlines (Real-Time Constraints)**
  - Responses must happen within a fixed time
  - Missing deadlines can cause system failure
  - Timing is often more important than speed
- Example:
  - A motor control system must update PWM signals at precise intervals to maintain stability.

Embedded Systems: Reactive Behaviour Real-World Examples

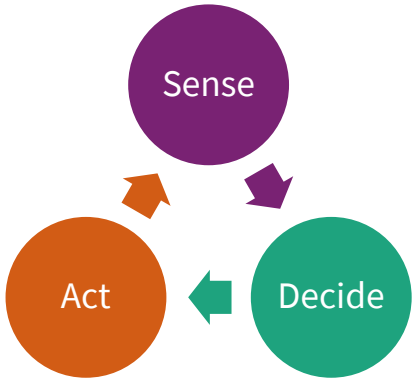
- **Vehicle Braking Systems**
  - Monitor wheel speed sensors
  - Detect slip
  - Adjust braking pressure in milliseconds
  - Must operate deterministically
- Failure to react on time can cause accidents.

- **Smart Appliances**
  - Detect button presses
  - Monitor internal temperature
  - Control heating elements
  - Shut down safely if faults occur
- System reacts to user input and internal state.

- **Industrial Control Systems**
  - Monitor pressure, temperature, flow rates
  - Control valves and motors
  - Maintain safe operating limits
  - Operate 24/7
- Real-time response ensures safety and efficiency.
  - precise intervals to maintain stability.

Embedded Systems: Reactive Behaviour

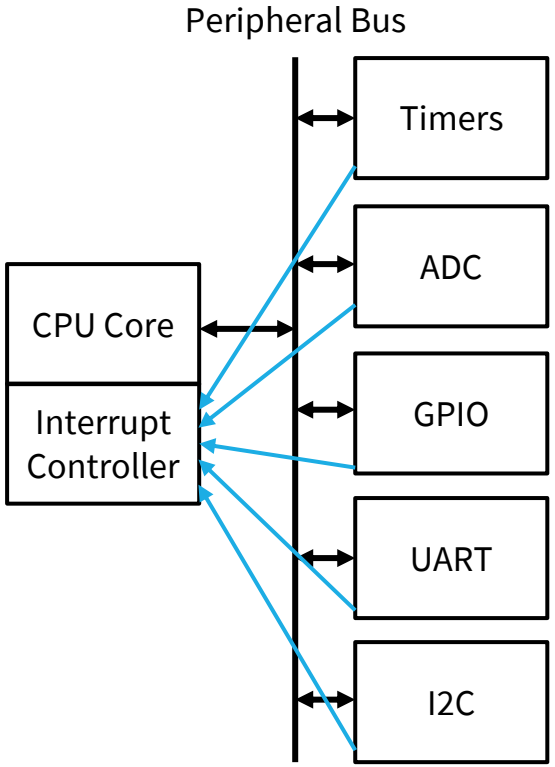
- Embedded systems are event-driven and time-sensitive.
  They must



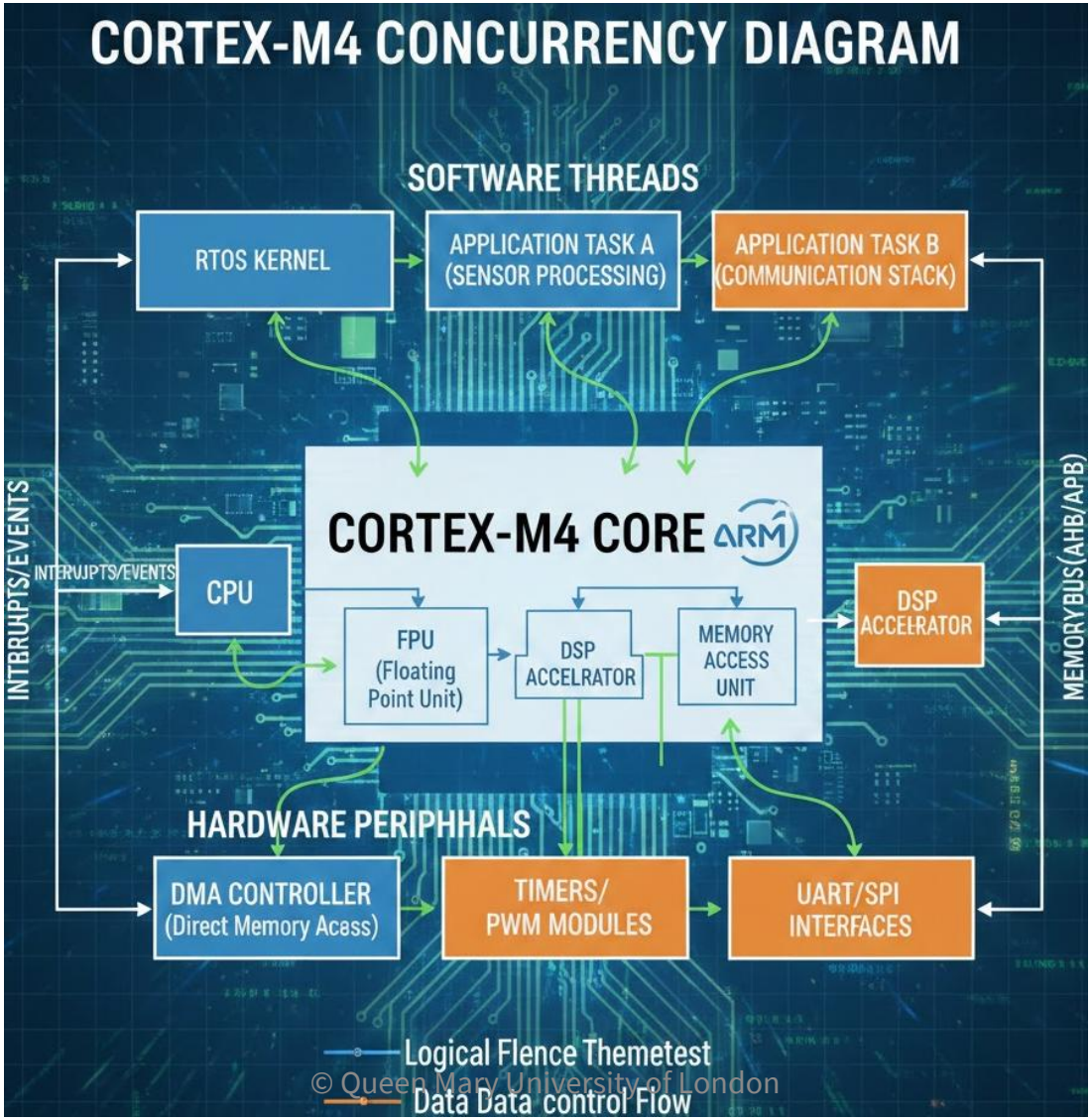
  continuously and reliably.

Hardware & Software: Concurrency

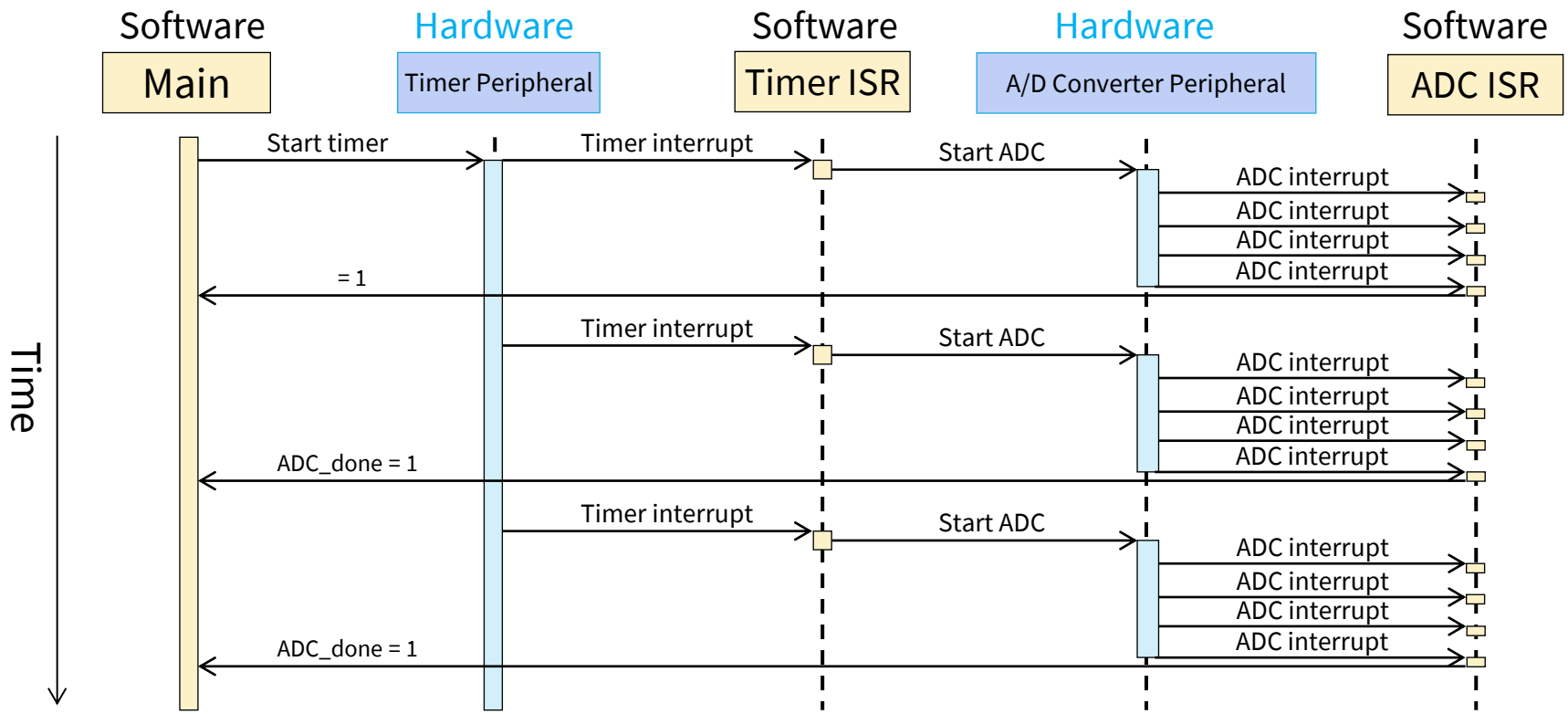
- CPU executes instructions from one or more thread of execution
- Specialised hardware peripherals add dedicated concurrent processing
  - Watchdog timer
  - Analog interfacing
  - Timers
  - Communications with other devices
  - Detecting external signal events
  - LCD driver
- Peripherals use interrupts to notify CPU of events



Hardware & Software: Concurrency



Concurrent Hardware & Software Operation



- Embedded systems rely on both hardware peripherals and software (running on MCU) to get everything done on time

Embedded Systems: Attributes (3)

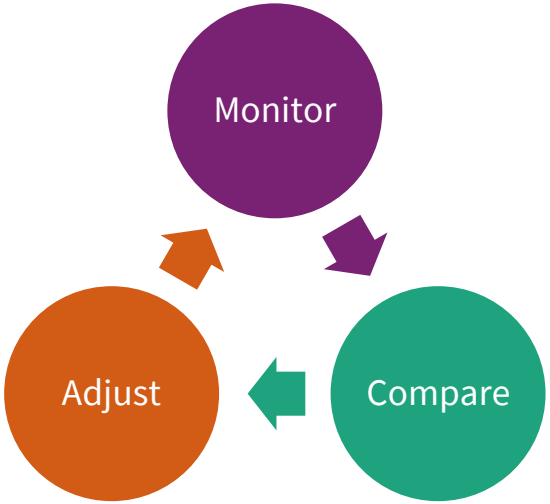
Fault handling & Diagnostics

- Many systems must operate independently for long periods of time, requiring system to handle likely faults without crashing.
- Often fault-handling code is larger and more complex than the normal-case code.

- Logs/records stored in memory/disk help service personnel determine problem quickly

Embedded System: Applications

- Closed-loop control system
  - Monitor a process, compare to threshold, adjust an output to maintain desired set point (temperature, speed, direction, etc.)
- Sequencing
  - Step through different stages based on environment and system
- Signal processing
  - Remove noise, select desired signal features
- Communications and networking
  - Exchange information reliably and quickly



Discussion: Identify embedded systems that realise the above-mentioned functions.

Benefits of Embedded Systems

- Greater performance and efficiency
  - Software makes it possible to provide sophisticated control
- Lower costs
  - Less expensive components can be used
    e.g. standard LCD display that works with standard MCUs
  - Manufacturing costs reduced – mass production
  - Maintenance costs reduced – fewer components
- Better dependability
  - Adaptive system which can compensate for failures
    e.g. when the motor is overheated, slow it down
  - Better diagnostics (e.g. logging) to improve repair time

Embedded Design: Constraints

- Cost

  Competitive markets penalise products which don't deliver adequate value for the cost
- Computation
  - Memory (RAM), code space (ROM or flash), processor cycles or speed, power consumption (see below), processor peripherals
- Size and weight limits
  - Mobile (aviation, automotive) and portable (e.g. handheld) systems
- Power and energy limits
  - Battery capacity
- Environment
  - Temperatures may range from -40°C to 125°C, or even more
  - Cooling limits

Impact of Constraints

- Microcontrollers (rather than microprocessors)
  - Peripherals are included to interface with other devices, respond efficiently.
  - On-chip RAM, ROM reduce circuit board complexity and cost.
- Programming language
  - Programmed in **C** rather than Java (smaller and faster code, so less expensive MCU)
  - Some performance-critical code may be in **assembly language**
- Operating system (OS)
  - Typically, no OS, but instead simple scheduler (or even just interrupts + main code (foreground/background system)
  - If OS is used, likely to be a Real-Time OS with a small code size

Summary

- We define that an embedded system is a computerised system that is purpose-built for its application.
- Examples of typical embedded systems are discussed with reference to three common attributes.
- (Micro)processor/Microcontroller is the heart of modern embedded systems, and it dominates the design cycle of the system.
- Identifying the constraints of your system is a key first step in designing a good embedded systems.